

ByteBuf之一

ByteBuf概述

北京理工大学计算机学院
金旭亮



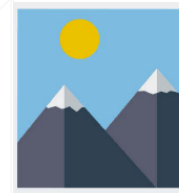


ByteBuffer是Netty的数据容器

ByteBuffer对象提供了相应的方法，能够直接存储8种基础数据类型：byte、boolean、char、short、int、long、float、double。

在Netty应用中，所有在网络中传输的数据，最终都是放到ByteBuffer中的，因此，基于Netty开发网络应用，必需掌握ByteBuffer的用法。

ByteBuffer与ByteBuf



网络中数据传输的基本单位是字节。Java NIO中提供有一个ByteBuffer用于为字节数据提供一个容器，但过于复杂，不便于使用。



Netty设计了一种新的数据容器—ByteBuf，既解决了ByteBuffer的局限性，又为网络应用程序的开发者提供了更易用的并且功能更为强大的API。



在Netty应用中，ByteBuffer与ByteBuf都可用，但建议全面采用ByteBuf，把ByteBuffer给“忘掉”，因为这两者“太像了”，混用容易搞混，除了加重心智负担，就没有什么其他的好处。

Netty内存管理-1



现代操作系统，普遍采用以下方式管理内存（称为jemalloc）：

- ▶ 首先，分配一块较大的内存空间。
- ▶ 然后，在内存分配和内存回收的操作过程中，会使用一个类似数据库表结构的记录来监控内存的使用状态，实时刷新。分配内存时，查表找没有被使用的内存进行分配，回收时，将释放的内存加入到“可用内存”中。
- ▶ 最后，当内存分配操作完成或内存被释放后，会同步更新这个内存状态记录。

上述方案历经多年实践、内存分配与回收算法持续优化，已经相当成熟，Netty的内存管理机制，借鉴了上述思路。

Netty内存管理-2



Netty框架的内存管理包括“有缓冲池”和“无缓冲池”两种方式，后者又可进一步细分为Unsafe和Direct两种子模式。

有缓冲池

在内存回收时，将信息记录在“缓冲池（pool）”中，下次如果有合适的分配请求，直接从缓冲池中复用。

Unsafe

是JDK底层的一个负责I/O操作的对象，可以直接基于内存地址进行读写操作。

Direct

堆外内存，直接调用JDK的底层API进行物理内存分配，不在JVM的堆内存中，需要手动释放。



实践中，使用“有缓冲池”的内存管理方式相比于“无缓冲池”的方式，其内存分配和内存回收的工作效率会更高，因此，它是Netty默认采用的方式。

Netty中让人眼花缭乱的ByteBuf类型

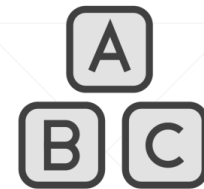


如前所述，Netty中混合采用了三种内存管理方式，这三种方式相互搭配，得到了若干种ByteBuf类型，各有各的适用场景。

类	解 释
PooledHeapByteBuf	池化的堆内缓冲区
PooledUnsafeHeapByteBuf	池化的Unsafe堆内缓冲区
PooledDirectByteBuf	池化的直接（堆外）缓冲区
PooledUnsafeDirectByteBuf	池化的Unsafe直接（堆外）缓冲区
UnpooledHeapByteBuf	非池化的堆内缓冲区
UnpooledUnsafeHeapByteBuf	非池化的Unsafe堆内缓冲区
UnpooledDirectByteBuf	非池化的直接（堆外）缓冲区
UnpooledUnsafeDirectByteBuf	非池化的Unsafe直接（堆外）缓冲区

表中列出了Netty所定义的N多种ByteBuf，但其中在多数开发场景中，只会用到少数几种，因此，对于多数ByteBuf类型，有所了解就OK了。

实际开发中ByteBuf的常用使用模式



堆缓冲模式

将数据存取在JVM堆内的一个字节数组中，这种模式被称为“支撑数组（backing array）”，为业务逻辑提供数据。

直接缓冲区模式

JVM支持在堆外保存数据，这种数据存储区，被称为“直接缓冲区”，其内存其实是由操作系统管理的，通常用它来从底层Socket中读取数据，之后，再将其复制到JVM堆中，以实现各种业务逻辑。这种适合于从网络直接输入和输出的场景。

复合缓冲区模式

如果已经有了现成的ByteBuf，则可以把它们“组装”为一个“复合”的缓冲区，无需进行数据复制，即可通过复合缓冲区直接访问数据，比如，使用它来存储HTTP响应的header和body数据。



这三种模式，Netty都提供简单方法可供直接创建其实例。

ByteBuf的基本编程模式



创建实例

使用特定类型的
ByteBufAllocator创建

01



02



03



数据读取

read系列方法会改变读取
指针，get系列方法不会

数据写入

write系列方法会改变读取
指针，set系列方法不会

从下一讲开始，我们将围绕着这个编程模式来介绍ByteBuf的基本用法。